AWS PRODUCTION ARCHITECTURE PATTERNS

Complete Guide to Production-Grade Deployment Patterns

ALB • ECS • EKS • API Gateway • CloudFront • Lambda

With Layman Explanations, Architecture Diagrams & Interview Tips

Interview & System Design Preparation

TABLE OF CONTENTS

1. Production Grade Assessment

An honest assessment of which deployment patterns are truly production-grade, which are possible but rare, and which should be avoided.

Pattern	Production?	Used By
ALB → ECS (Fargate)	■ YES	Most companies
ALB → EKS	■ YES	Large companies
API Gateway → Lambda	■ YES	Serverless apps
API GW → ECS (via ALB)	■ YES	REST API products
CloudFront → ALB → ECS	■ YES	Global apps
CF → S3 + API GW → Lambda	■ YES	Static + API
NLB → ECS/EKS	■ YES	gRPC, high perf
API GW → ECS (direct)	■■ Possible	Rare, limited
EC2 bare metal	■ Legacy	Old companies
Docker Compose in prod	■ Not recommended	Dev/staging only
Single server	■ Not scalable	Side projects

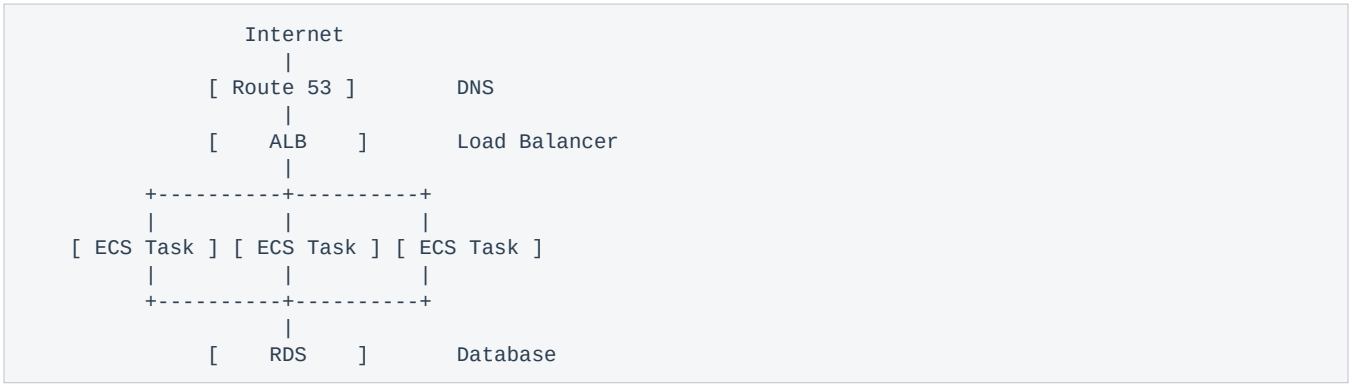
2. Pattern 1: ALB → ECS Fargate

MOST COMMON PATTERN — Used by the majority of companies deploying containerized apps on AWS.

Layman Explanation

Think of it like a restaurant chain. The ALB is the receptionist who sends customers to available tables. ECS is the kitchen manager who decides which chefs are working. Fargate is a rented kitchen — you do not own the building, you just cook. Each Task is an individual chef (your running container).

Architecture



Step-by-Step Setup

- **Step 1: Containerize** — Write Dockerfile, build image, push to ECR
- **Step 2: Create ECS Cluster** — Choose Fargate (no EC2 to manage)
- **Step 3: Task Definition** — Image, CPU, memory, ports, env vars, IAM roles
- **Step 4: ECS Service** — Desired count, auto-scaling rules
- **Step 5: Create ALB** — Target Group (IP type), health check path
- **Step 6: Connect ALB to ECS** — ECS auto-registers task IPs
- **Step 7: Auto-Scaling** — CPU > 70%, min: 2, max: 10 tasks

VPC Layout

```
VPC
|-- Public Subnets
|   |-- ALB
|   +-- NAT Gateway
|-- Private Subnets
|   |-- ECS Task 1, 2, 3
+-- Database Subnet
    +-- RDS (Multi-AZ)
```

Why This is Production Grade

- ■ Auto-scaling handles traffic spikes automatically
- ■ Self-healing — unhealthy tasks replaced automatically
- ■ No server management (Fargate = serverless containers)

- ■ Multi-AZ — survives data center failures
- ■ Rolling deployments — zero downtime
- ■ Health checks — ALB removes unhealthy containers
- ■ Cost-effective — pay per task, not per idle server

3. Pattern 2: ALB → EKS (Kubernetes)

FOR LARGE ORGANIZATIONS — Same concept as ECS but uses Kubernetes. More powerful, more complex.

ECS is like an automatic car (simpler, AWS manages more). EKS is like a manual car (more control, you manage more). Use EKS when your team knows Kubernetes or you need multi-cloud portability.

ECS vs EKS Decision

Factor	Use ECS	Use EKS
Team expertise	AWS-focused	Knows Kubernetes
Cloud strategy	AWS only	Multi-cloud
Complexity	Simpler	More control
Microservices	< 20 services	50+ services
Team size	Small-medium	Large team

Kubernetes Key Concepts

Concept	What It Is	Analogy
Pod	Smallest unit (1+ containers)	A room with your app
Deployment	Manages pods count/version	Manager: "keep 3 rooms"
Service	Network endpoint	Phone number for rooms
Ingress	External access (ALB)	Reception desk
Namespace	Logical separation	Floors in a building

Sample Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: my-app
          image: 123456.dkr.ecr.../my-app:latest
          ports: [containerPort: 8080]
          resources:
            requests: {cpu: 250m, memory: 512Mi}
            limits:   {cpu: 500m, memory: 1Gi}
          readinessProbe:
            httpGet: {path: /actuator/health, port: 8080}
```

4. Pattern 3: API Gateway → Lambda

FULLY SERVERLESS — No servers. Code runs only when called. Pay zero when idle.

Traditional servers are like owning a car — you pay even when parked. Lambda is like Uber — you pay only when you ride. Code runs on demand and scales automatically.

Architecture

```
Client -> API Gateway -> Lambda Function -> DynamoDB/RDS
                                         -> S3
                                         -> SQS/SNS

API Gateway:              Lambda:
  Auth, Rate limit          Your code
  Throttle, Cache           Auto-scales
                            Pay per invocation
```

When to Use / Not Use

Use Lambda When	Do NOT Use When
Low/unpredictable traffic	High-traffic (cold starts)
Event-driven processing	Long processes (>15 min)
Simple CRUD APIs	WebSocket/real-time apps
Scheduled background tasks	Persistent connections
Cost-sensitive projects	Heavy computation

5. Pattern 4: API Gateway → ALB → ECS

PUBLIC APIs WITH MANAGEMENT — API Gateway handles security, ALB distributes to containers.

API Gateway is the security guard (auth, rate limiting, API keys). ALB is the floor manager (distributes to workers). ECS tasks are the workers doing the actual job.

Architecture

```
Client -> API Gateway -> VPC Link -> ALB (private) -> ECS Tasks

API Gateway adds:           ALB provides:
  API key management          Load balancing
  Rate limiting                Health checks
  Request transformation       SSL termination
  Caching, WAF                 Connection draining
  Usage plans, Dev portal
```

Connection: VPC Link

API Gateway is public; ALB is private inside VPC. VPC Link creates a private tunnel between them. Without it, ALB would need to be public, letting hackers bypass all API Gateway security.

VPC Link v2 (recommended) connects directly to ALB via HTTP API. VPC Link v1 (legacy) requires an extra NLB in between. Always use v2.

Security Chain

```
API Gateway -> VPC Link SG -> ALB SG -> ECS SG -> RDS SG
Each layer only trusts the one before it.
```

6. Pattern 5: CloudFront → ALB → ECS

GLOBAL APPLICATIONS — CDN in front for worldwide speed and DDoS protection.

Without CloudFront, a user in India calling a US server faces high latency. CloudFront has edge locations worldwide. The user is served from the nearest edge (e.g., Mumbai), making it much faster.

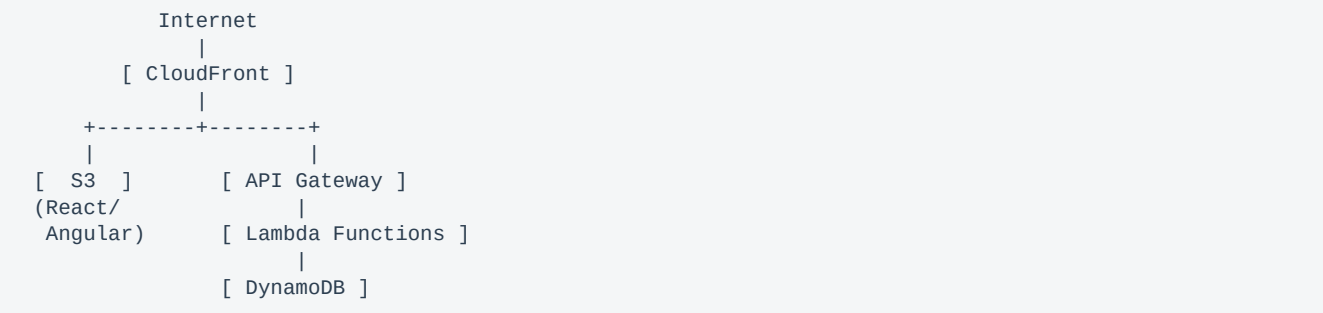
What CloudFront Adds

- ■ Global caching — static content from nearest edge
- ■ DDoS protection — AWS Shield built-in
- ■ SSL termination — HTTPS everywhere
- ■ Compression — gzip/brotli for faster transfers
- ■ Geo-restriction — block specific countries
- ■ Lambda@Edge — custom logic at the edge

7. Pattern 6: CloudFront → S3 + Lambda

FULL SERVERLESS STACK — Static site on S3, API on Lambda. Cost: ~$1-5/month for low traffic.

Architecture



Setup Steps

- **1.** Build React/Angular → deploy to S3
- **2.** CloudFront serves static files globally
- **3.** API calls route to API Gateway
- **4.** API Gateway triggers Lambda functions
- **5.** Lambda reads/writes to DynamoDB
- **6.** Everything auto-scales, zero servers

8. Pattern 7: NLB → ECS/EKS

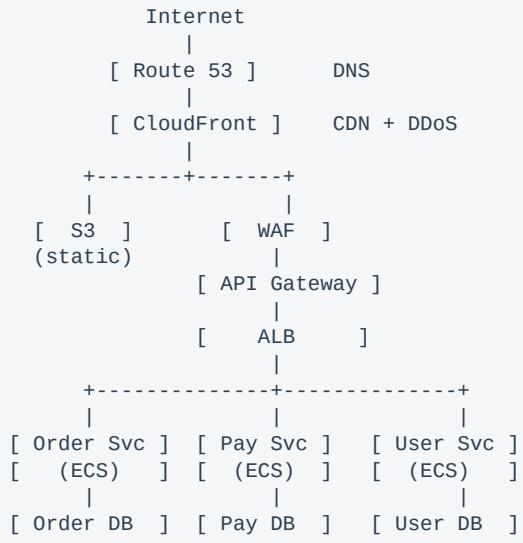
HIGH PERFORMANCE — Network Load Balancer for ultra-low latency and high throughput.

ALB operates at Layer 7 (HTTP) — reads requests, routes by URL. NLB operates at Layer 4 (TCP/UDP) — just forwards packets without reading, making it extremely fast.

Use NLB	Use ALB
gRPC services	REST APIs
Real-time gaming	Web applications
IoT data streams	Path-based routing
Ultra-low latency	Content-based routing
Millions of req/sec	WebSocket support
Static IP required	Host-based routing

9. Full Production Architecture

COMPLETE ENTERPRISE SETUP — All patterns combined for production microservices.



Supporting Services

- **ElastiCache (Redis)** — Caching + session store
- **SQS / SNS** — Async messaging
- **CloudWatch** — Monitoring + logging
- **Secrets Manager** — Credentials management
- **ECR** — Container image registry

- **CodePipeline** — CI/CD automation
- **X-Ray** — Distributed tracing

10. Comparison & Decision Flow

Pattern	Best For	Cost	Complexity
ALB → ECS Fargate	Most apps, startups	Medium	Low
ALB → EKS	Large orgs, multi-cloud	High	High
API GW → Lambda	Low traffic, events	Very Low	Low
API GW → ALB → ECS	Public APIs, SaaS	Medium+	Medium
CF → ALB → ECS	Global apps	Medium	Medium
CF → S3 + Lambda	Full serverless	Very Low	Medium
NLB → ECS/EKS	gRPC, low latency	Medium	Medium

Decision Flow

```
START
  |
  +-- Need servers?
      |-- NO -> Serverless
      |       |-- Simple API? -> API GW + Lambda
      |       +-- Full app?   -> CF + S3 + Lambda
      |
      +-- YES -> Containers
              |-- Knows K8s?
              |   YES -> EKS  |  NO -> ECS Fargate
              |
              |-- Need API mgmt (keys, throttle)?
              |   YES -> API GW -> ALB -> ECS
              |   NO  -> ALB -> ECS
              |
              |-- Global audience?
              |   YES -> CloudFront -> ALB -> ECS
              |
              +-- gRPC / ultra-low latency?
                  YES -> NLB -> ECS
                  NO  -> ALB -> ECS
```

11. Deep Dive: VPC Link

The Core Problem

API Gateway lives outside your VPC (public AWS service). ALB lives inside your VPC (private). They cannot talk directly. VPC Link creates a private tunnel that bridges this gap.

Why Not Just Make ALB Public?

■ **Public ALB (No VPC Link):** Anyone who discovers your ALB URL can hit it directly, bypassing all API Gateway security. Rate limiting, API keys, JWT auth — all useless.

■ **Private ALB (With VPC Link):** ALB is hidden from internet. Only reachable through API Gateway. All security enforced at single entry point. No bypass possible.

Three Reasons for VPC Link

- **Security:** ALB stays private, not exposed to internet
- **Network Privacy:** Traffic stays on AWS private network, never public internet
- **Single Entry Point:** All traffic through API Gateway — no bypass

Setup Steps

- **Step 1:** Create PRIVATE (internal) ALB in private subnets
- **Step 2:** Create VPC Link v2 pointing to your VPC
- **Step 3:** Create HTTP API in API Gateway
- **Step 4:** Add integration: Private Resource → ALB via VPC Link
- **Step 5:** Configure routes (ANY /api/{proxy+})

Request Flow

```
1. User calls: GET https://api.myapp.com/api/employees
2. Route 53 -> API Gateway
3. API Gateway: validates API key, rate limit, JWT
4. Forwards through VPC Link (private tunnel)
5. ALB receives, checks health, routes to ECS task
6. ECS task processes, queries DB, returns response
7. Response: ECS -> ALB -> VPC Link -> API GW -> User
```

12. Deep Dive: Target Groups in Fargate

The Question

In Fargate there are no fixed servers or IPs. Tasks come and go dynamically. So how does ALB know where to send traffic?

> **Answer:** You create an EMPTY target group (IP type). The ECS Service automatically registers and deregisters task IPs as tasks start, stop, scale, or get replaced. You never add targets manually.

Target Group Types

Type	Used For	Fargate?
Instance	EC2 instances	■ NO
IP ■	IP addresses (ENI)	■ YES
Lambda	Lambda functions	■ Not relevant

Automatic Registration Timeline

```
BEFORE:  Target Group: [ EMPTY ]

Task 1 starts (IP: 10.0.1.45):
  -> ECS registers 10.0.1.45
  -> TG: [ 10.0.1.45 ]

Task 2, 3 start:
  -> TG: [ 10.0.1.45, 10.0.2.67, 10.0.1.89 ]

Task 2 UNHEALTHY:
  -> ALB removes 10.0.2.67
  -> ECS starts Task 4 (10.0.3.12), registers it
  -> TG: [ 10.0.1.45, 10.0.1.89, 10.0.3.12 ]

AUTO-SCALE (traffic spike):
  -> Task 5 added (10.0.2.99)
  -> TG: [ 10.0.1.45, 10.0.1.89, 10.0.3.12, 10.0.2.99 ]
```

Rolling Deployment (Zero Downtime)

```
v1 -> v2 deployment:
  [v1, v1, v1]            start
  [v1, v1, v1, v2]        new v2 added + health checked
  [v1, v1, v2]            old v1 drained + removed
  [v1, v2, v2]            repeat...
  [v2, v2, v2]            done! zero downtime
```

You vs ECS Responsibilities

You Configure	ECS Does Automatically
Create empty target group (IP)	Register task IPs

You Configure	ECS Does Automatically
Set health check path	Deregister failed tasks
Create ALB + listeners	Register on scaling
Create ECS Service	Replace unhealthy tasks
Link service to TG	Rolling update management

13. Interview-Ready Answers

Q: Explain your deployment pattern.

"We use ALB with ECS Fargate. Each microservice is containerized, pushed to ECR, and deployed as ECS tasks behind an ALB. We chose Fargate because we did not want to manage servers. Auto-scaling handles traffic spikes, and rolling deployments give us zero downtime."

Q: How does API Gateway connect to a private ALB?

"Through a VPC Link, which creates a private tunnel from API Gateway into the VPC. This keeps the ALB private so no one can bypass API Gateway's security. We use VPC Link v2 with HTTP API for direct ALB connection without needing an NLB."

Q: How does ALB route to Fargate tasks if there are no servers?

"We create an empty target group with IP type. When the ECS Service launches tasks, each task gets its own ENI with a private IP. ECS automatically registers these IPs with the target group. When tasks stop or become unhealthy, ECS deregisters them. We never manage targets manually."

Q: How would you choose between these patterns?

"For most apps: ALB + ECS Fargate. For public APIs needing rate limiting: add API Gateway with VPC Link. For global reach: add CloudFront. For Kubernetes teams or multi-cloud: use EKS. For low-traffic or event-driven: go fully serverless with Lambda."

Q: Why VPC Link instead of a public ALB?

"A public ALB means hackers can discover the URL and bypass API Gateway entirely — making rate limiting, API keys, and JWT auth useless. VPC Link keeps ALB private so the ONLY way to reach it is through API Gateway. Security is enforced at a single entry point."